

Assignment 3 | Bridge Pattern

Student: Dias Sabit

Group: SE-2526

Course: ShP-2216 Software Design Patterns

Topic: A — Drawing

Language: Java, JDK 17

GitHub Repository: <https://github.com/diassabit1/SDP-Bridge-pattern>

Base Commit: e6250f8

Submitted Commit: 8bab1005e5c6aed2c5ea9fea07c16579383619f2

1. Design and Bridge Rationale

This project demonstrates the **Bridge design pattern** using the Drawing topic.

The application contains two independently varying dimensions:

1. **Abstraction hierarchy:** Shape → Circle, Square
2. **Implementation hierarchy:** Renderer → VectorRenderer, RasterRenderer, AsciiRenderer

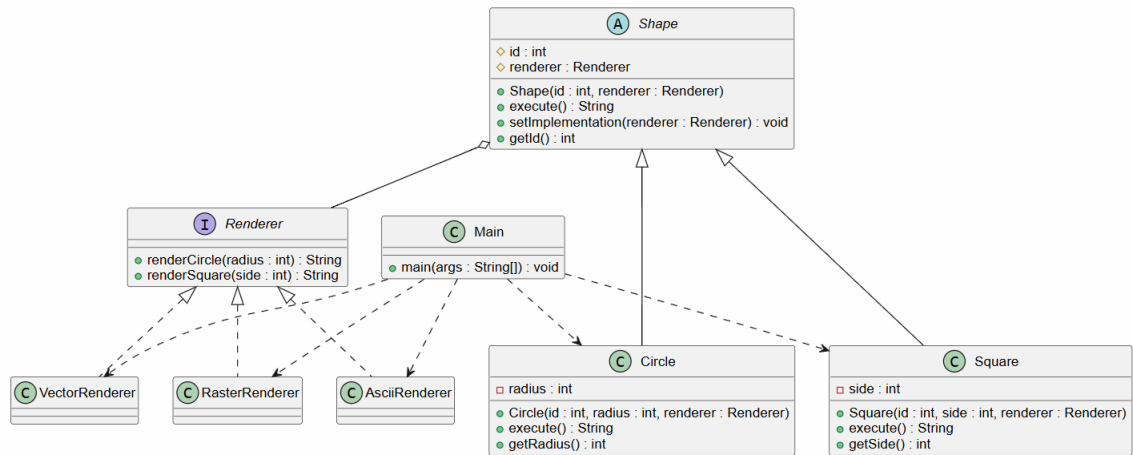
The Shape abstraction stores a reference to the `Renderer` interface. This reference is supplied through the constructor and can later be replaced using `setImplementation(Renderer)`.

The Bridge pattern is appropriate because the shape type and rendering method can vary independently. A `Circle` does not need to know whether it is rendered using vector, raster, or ASCII rendering. It only delegates the rendering operation to the `Renderer` interface.

This design also allows the same `Circle` object to change its implementation at runtime. In T5, the object remains the same while its renderer changes from `VectorRenderer` to `RasterRenderer`.

2. UML Class Diagram

The UML diagram represents the following structure:



The most important Bridge connection is the interface-typed field:
`protected Renderer renderer;`

The abstraction hierarchy therefore depends on the `Renderer` interface rather than on any concrete renderer.

3. Role Map

Role	Class	Source
Abstraction	Shape	src/Shape.java
A1	Circle	src/Circle.java
A2	Square	src/Square.java
Implementor	Renderer	src/Renderer.java
I1	VectorRenderer	src/VectorRenderer.java
I2	RasterRenderer	src/RasterRenderer.java
I3	AsciiRenderer	src/AsciiRenderer.java
Client	Main	src/Main.java

Shape contains the bridge reference, `execute()` is the public abstraction operation, and `setImplementation(Renderer)` performs runtime implementation replacement.

4. Clean Code Evidence

4.1 Separation of Responsibilities

```

public class Circle extends Shape {
    private final int radius;

    @Override
    public String execute() {
        return renderer.renderCircle(radius);
    }
}
  
```

`Circle` is responsible for circle-specific data and behavior. Rendering details are delegated to the renderer. This separates abstraction responsibilities from implementation responsibilities.

4.2 Meaningful Names

```
Renderer vectorRenderer = new VectorRenderer();
Renderer rasterRenderer = new RasterRenderer();
Renderer asciiRenderer = new AsciiRenderer();
```

The variable names clearly describe the implementation being used. This improves readability and makes the role of each renderer immediately understandable.

4.3 Small Focused Methods

```
@Override
public String renderSquare(int side) {
    return "VECTOR square side=" + side;
}
```

The method performs one focused task: it creates the rendering description for a square. It does not contain unrelated logic.

4.4 Avoidance of Duplication

```
public abstract class Shape {
    protected Renderer renderer;

    public void setImplementation(Renderer renderer) {
        this.renderer = renderer;
    }
}
```

The common bridge state and runtime-switch operation are implemented once in the base abstraction instead of being duplicated in `Circle` and `Square`.

4.5 Encapsulation of State

```
public class Circle extends Shape {
    private final int radius;

    public int getRadius() {
        return radius;
    }
}
```

The radius is private and final. It is initialized through the constructor and cannot be directly modified from outside the class. This protects the domain state of the abstraction.

5. Required Demonstration Checks

The application provides a non-interactive `--demo` mode and executes all seven checks in the required order.

Check	Setup / Action	Expected Result
-------	----------------	-----------------

Check	Setup / Action	Expected Result
T1	Circle + VectorRenderer	VECTOR circle radius=2
T2	Circle + RasterRenderer	RASTER circle radius=2
T3	Square + VectorRenderer	VECTOR square side=3
T4	Square + RasterRenderer	RASTER square side=3
T5	Same Circle: Vector → Raster	Same object and unchanged state
T6	Circle + AsciiRenderer	ASCII circle radius=2
T7	Square + AsciiRenderer	ASCII square side=3

The checks compare actual application results with expected values. The summary is calculated from the number of passed checks.

6. Runtime Switching in T5

T5 demonstrates the most important runtime property of the Bridge implementation.

The same `Circle` object is created using `VectorRenderer`:

```
Circle circle = new Circle(10, 2, firstRenderer);
Shape originalReference = circle;
```

```
String before = circle.execute();
```

The implementation is then replaced:

```
circle.setImplementation(secondRenderer);
```

```
Shape afterReference = circle;
String after = circle.execute();
```

The test verifies object identity using reference equality:

```
boolean sameObject = originalReference == afterReference;
```

It also verifies that the abstraction-side state is unchanged:

```
boolean stateUnchanged =
    idBefore == idAfter &&
    radiusBefore == radiusAfter;
```

Therefore, T5 proves that the implementation reference changes while the abstraction object, its ID, and its radius remain unchanged.

7. Independent Extension

The required extension was completed in two commits.

The first commit is the base implementation:

Base commit: e6250f8

It contains the working two-by-two solution with:

- Circle
- Square
- Renderer
- VectorRenderer
- RasterRenderer
- T1–T5

The second commit adds the independent implementation:

Extension commit: 8bab1005e5c6aed2c5ea9fea07c16579383619f2

The extension introduces:

```
public class AsciiRenderer implements Renderer {  
  
    @Override  
    public String renderCircle(int radius) {  
        return "ASCII circle radius=" + radius;  
    }  
  
    @Override  
    public String renderSquare(int side) {  
        return "ASCII square side=" + side;  
    }  
}
```

It also updates `Main` to demonstrate T6 and T7.

The existing abstraction classes, `Renderer` interface, `VectorRenderer`, and `RasterRenderer` were not modified by the extension. Therefore, the new implementation is independent of the existing hierarchies.

The source diff is generated from the base commit using:

```
git diff e6250f8 HEAD -- src > extension.diff
```

8. Bridge versus Adapter

The main difference between Bridge and Adapter is their intent.

Bridge separates two dimensions of variation so that both can evolve independently. In this project, shapes and rendering implementations are deliberately separated. `Circle` and `Square` depend on the `Renderer` abstraction rather than concrete renderer classes.

Adapter, in contrast, is normally used when an existing class has an incompatible interface and must be converted into an interface expected by the client.

This project does not need to convert an incompatible existing API. Instead, it intentionally designs the abstraction and implementation hierarchies separately from the beginning. Therefore, Bridge is the appropriate pattern.

9. Trade-off

One trade-off of Bridge is additional structural complexity. Instead of using one hierarchy, the solution contains an abstraction hierarchy, an implementor interface, and several implementation classes.

This introduces more classes and an additional abstraction boundary. However, the trade-off is useful because it provides independent variation, runtime implementation switching, and independent extension without modifying the existing abstraction classes.

10. Reproducibility

The project can be compiled and executed without an IDE or external dependencies using JDK 17:

```
javac --release 17 -encoding UTF-8 -d out "@sources.txt"
java -cp out Main --demo
```

The program accepts no interactive input in demo mode and calculates the final result from the seven checks.

Expected final summary:

```
SUMMARY: 7/7 PASS
```

11. Demo Evidence

The submitted implementation produces the following expected demonstration results:

```
T1 PASS | Circle + VectorRenderer | result=VECTOR circle radius=2
T2 PASS | Circle + RasterRenderer | result=RASTER circle radius=2
T3 PASS | Square + VectorRenderer | result=VECTOR square side=3
T4 PASS | Square + RasterRenderer | result=RASTER square side=3
T5 PASS | sameObject=true | stateUnchanged=true | before=VECTOR circle
radius=2 | after=RASTER circle radius=2
T6 PASS | Circle + AsciiRenderer | result=ASCII circle radius=2
T7 PASS | Square + AsciiRenderer | result=ASCII square side=3
SUMMARY: 7/7 PASS
```

12. Conclusion

The submitted application demonstrates the Bridge design pattern by separating the shape abstraction hierarchy from the renderer implementation hierarchy.

`Circle` and `Square` retain their own domain data, while `VectorRenderer`, `RasterRenderer`, and `AsciiRenderer` provide independently varying rendering behavior. The `Renderer` interface connects the two hierarchies through composition.

The T5 test demonstrates runtime switching of the implementation while preserving the same abstraction object and its domain state. The extension step adds `AsciiRenderer` without changing the existing abstraction hierarchy or the original implementations.

The final implementation therefore demonstrates the required Bridge structure, all required combinations, runtime switching, independent extension, reproducible execution, and the requested Clean Code practices.

References

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] Astana IT University, School of Software Engineering, *Assignment 3 | Bridge Pattern*, ShP-2216 Software Design Patterns, 2026–2027, Yerassyl Bekenov.
- [3] Refactoring.Guru, *Bridge Design Pattern*, <https://refactoring.guru/design-patterns/bridge>, accessed October 4, 2026.